

超大集群还是多集群? 如何定位关键瓶颈

徐俊杰 DaoCloud @pacoxu

郭奇峰 DaoCloud @cyclinder





A Huge Cluster or Multi-Clusters? Identifying the Bottleneck - Paco Xu & Saiyam Pathak



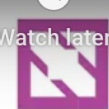
KubeCon



Watch later



Share



CloudNativeCon

Europe 2025

A Huge Cluster or Multi-Clusters? Identifying the Bottleneck



*Paco Xu, DaoCloud &
Saiyam Pathak, Loft Labs*

Watch on  YouTube



前情提要：去年 KubeCon 欧洲的演讲
- 后半部分内容包含了 vCluster 重点分析（本次不会详细展开）

Content 目录

01 常见问题

02 超大规模集群

03 多集群 / 多租户

04 网络



Part 01

常见问题

官方建议 + 常见痛点



[► Documentation](#)[▼ Getting started](#)[Learning environment](#)[► Production environment](#)[▼ Best practices](#)

Considerations for large clusters

[Running in multiple zones](#)[Validate node setup](#)[Enforcing Pod Security Standards](#)[PKI certificates and requirements](#)[► Concepts](#)

[Kubernetes Documentation](#) / [Getting started](#) / [Best practices](#)
/ [Considerations for large clusters](#)

Considerations for large clusters

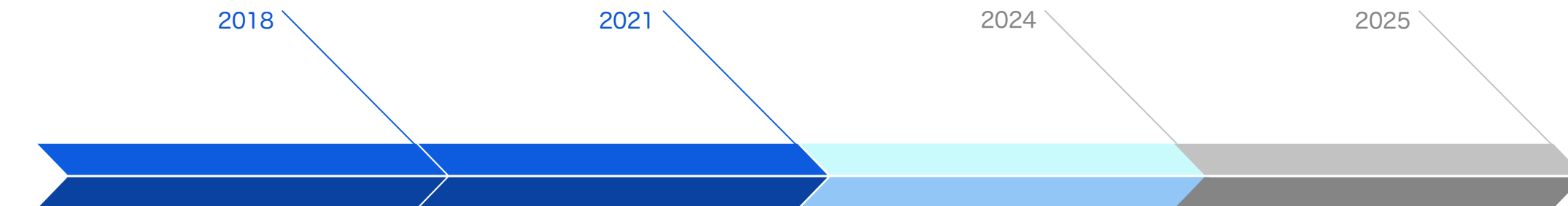
A cluster is a set of nodes (physical or virtual machines) running Kubernetes agents, managed by the control plane. Kubernetes v1.32 supports clusters with up to 5,000 nodes. More specifically, Kubernetes is designed to accommodate configurations that meet *all* of the following criteria:

- No more than 110 pods per node
- No more than 5,000 nodes
- No more than 150,000 total pods
- No more than 300,000 total containers

You can scale your cluster by adding or removing nodes. The way you do this depends on how your cluster is deployed.

[✎ Edit this page](#)[✎ Create child page](#)[🔗 Create an issue](#)[🖨 Print entire section](#)[Cloud provider resource quotas](#)[Control plane components](#)[etcd storage](#)[Addon resources](#)[What's next](#)

Previous Public Blogs



Earlier Days

OpenAI: Scaling Kubernetes to 2,500 nodes(2018)



beyond the defaults(5000)

Ant Group: Managing 10,000+ nodes(2019).

OpenAI: Scaling Kubernetes to 7,500 nodes(2021).

Bayer Crop Science: 15,000 nodes on Spanner(2020)



Bayer CropScience

replace ETCD

ByteDance: Using KubeBrain to scale to 20,000 nodes(2022).

Google Kubernetes Engine (GKE): Scaling to 65,000+ nodes(2022)



Cache(KEPs)

Under the Hood: Amazon EKS Ultra Scale Clusters. 10k nodes(2025)

How Google Does It: Building the largest known Kubernetes cluster, with 130,000 nodes



What's the Largest K8S Cluster You're Running?

- 你的痛点是什么? https://www.reddit.com/r/kubernetes/comments/1husfza/whats_the_largest_kubernetes_cluster_youre/

 r/kubernetes • 2mo ago • 

What's the Largest Kubernetes Cluster You're Running? What Are Your Pain Points?



138



32 comments

Award

Share

Edit post

Save



常见的痛点

集群各类资源的总量（节点数并不是唯一标准）

更新频率

大量的事件

集群自动伸缩

多租户

Iptables 和 Endpoints 同

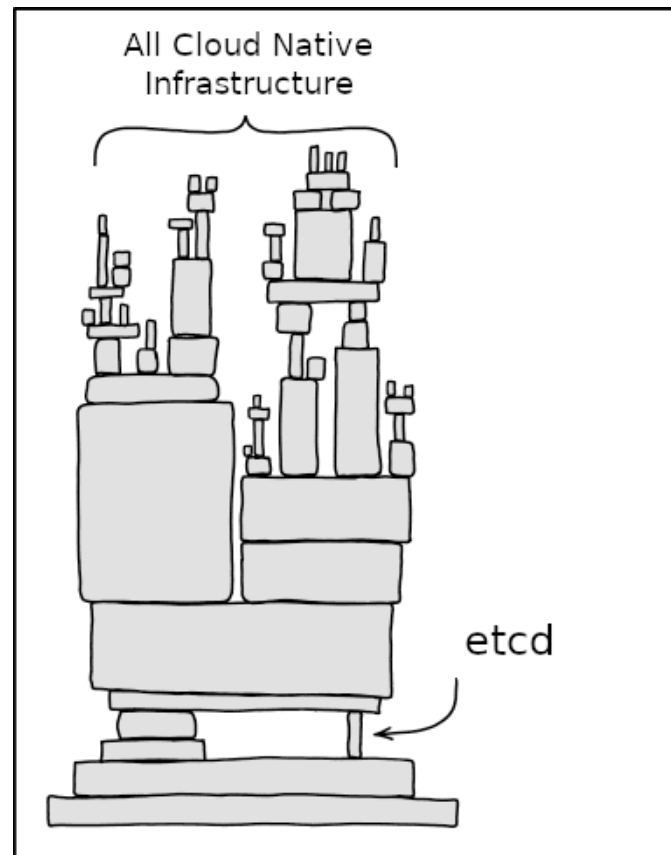
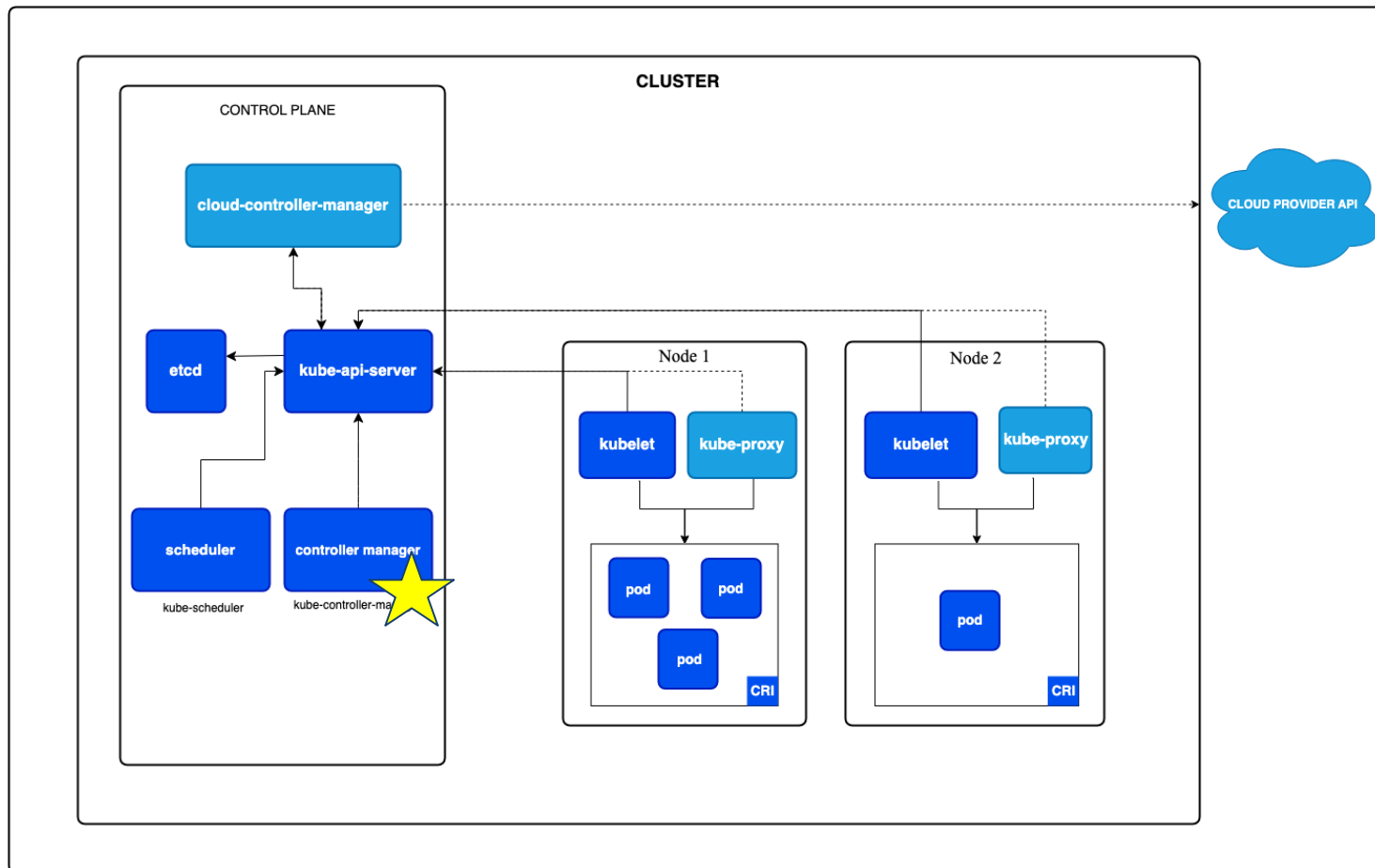
止
DNS

集群独立的健康比如
Prometheus

APIServer
DoS



常见瓶颈和问题



It's not DNS

There's no way it's DNS

It was DNS

Part 02

超大规模集群

大集群的常见瓶颈点和优化方案



超大规模集群

优点

- 降低维护成本
- 集中化管理，并可统一执行策略
- 优化资源利用率（例如可结合 VPA）
- 支持多可用区部署，避免单可用区故障



缺点

- 租户隔离较弱：主要依赖 namespace
- 故障爆炸半径较大
- 控制平面的性能下降
- 终端用户灵活性降低
 - 可选版本通常受限于 n 到 n-3
 - 扩展能力面临挑战
 - 需要定制化节点配置
 - 依赖 CRD 和 Operator，增加系统复杂度

1.32

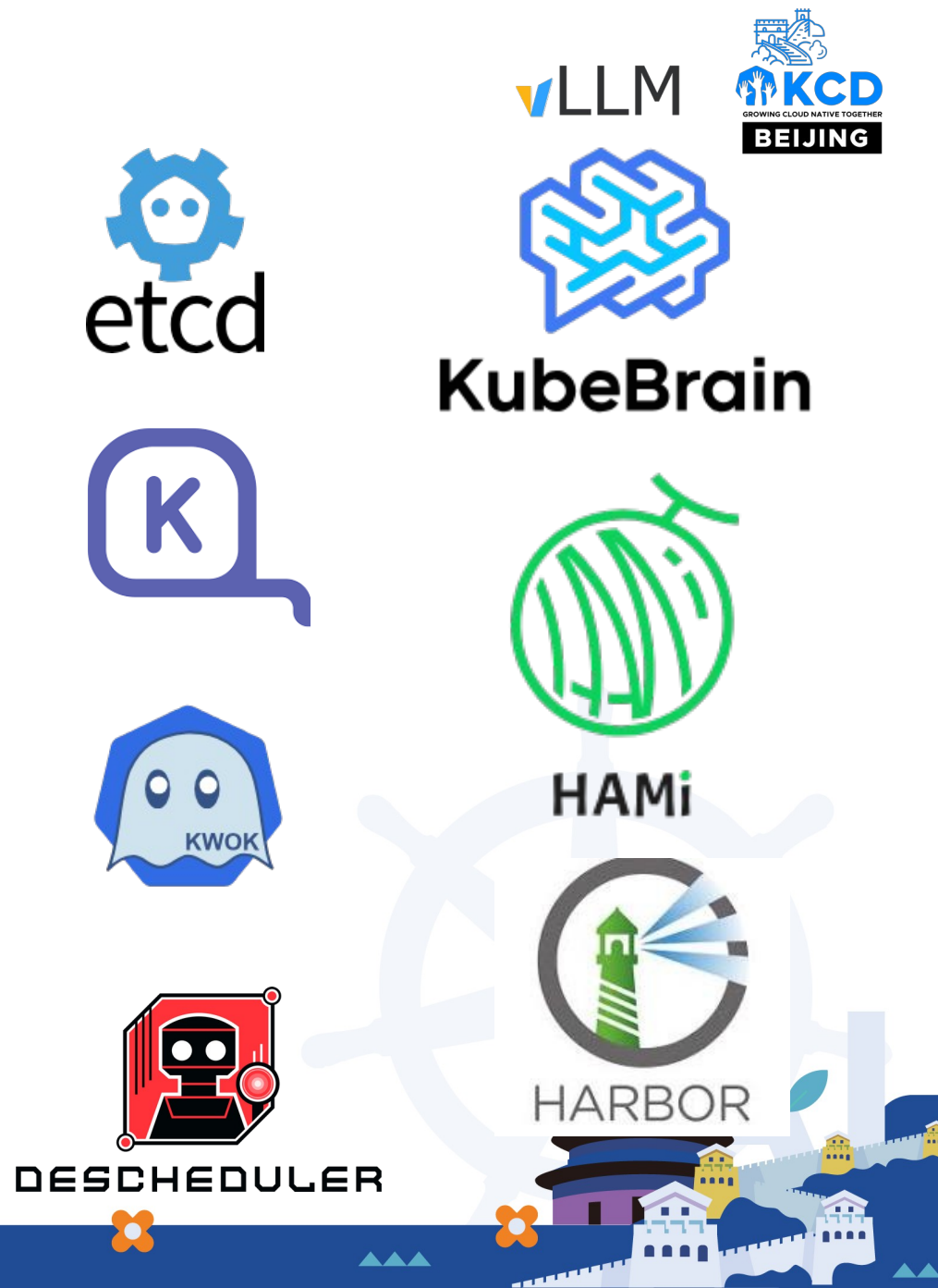
Kubernetes 1.32 于 Dec 28, 2025 进入维护模式。

1.32 生命周期结束于 Feb 28, 2026。



超大规模集群：挑战与解决方案

- API Server 压力：限流、APF 与缓存
 - 所有管理组件与 Operator 的性能
- 调度：调度器的选择与调优；Descheduler
 - 异构 CPU / GPU 架构
- ETCD
 - etcd 调优：heartbeat、snapshot
 - etcd 替代方案：KubeBrain（详见下一页）
 - etcd 读缓存：Clusterpedia
- 网络：
 - CNI、Gateway、DNS（node-local-dns）、NFTables 等
 - Pod / Service IP 范围，以及跨可用区问题
- 存储
- 自动扩缩容：Karpenter、KEDA
- 测试：KWOK、Kubemark
- 本地镜像仓库：Harbor Proxy Cache 或 CRProxy



Kubernetes 社区更新

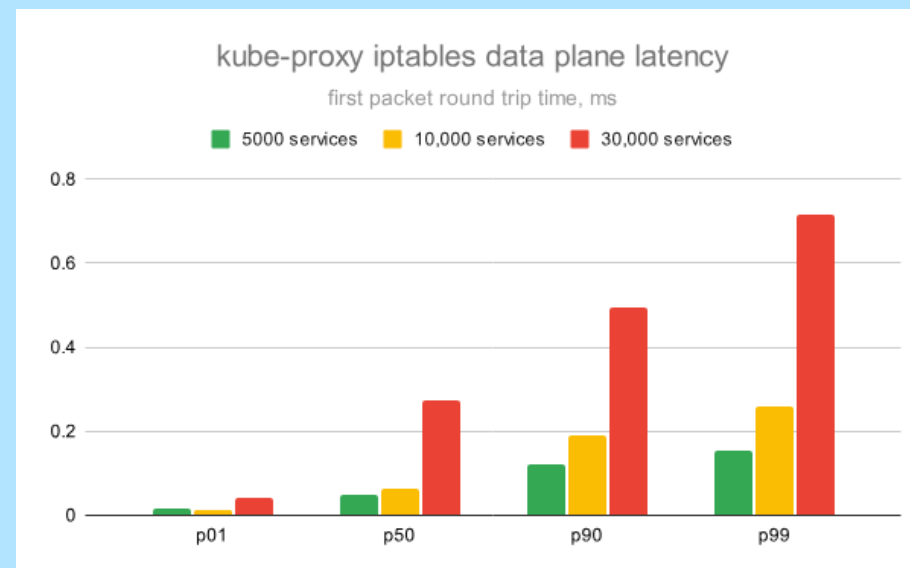
- `--etcd-servers-overrides` support (v1.1 @2015)
- KEP-0589: Efficient Node Heartbeats(GA in v1.17 @2019)
- KEP-0956: Watch bookmark(GA in v1.17 @2019)
- KEP-1152: Less object serializations(v1.17 @2019)
- KEP-0752: Endpoint Slices (GA in v1.21 @2021)
- KEP-1904: Efficient watch resumption after kube-apiserver reboot(GA in v1.24@2022)
- KEP-0365: Paginated API Lists(GA in v1.29 @2023)
- KEP-1040: Priority and Fairness for API Server Requests(GA in v1.29 @2023)
- **KEP-2340**: Consistent Reads from Cache(Beta in v1.31 @2024)
- KEP-3157: Allow informers for getting a stream of data instead of chunking.(Beta in v1.32@2024)
- KEP-4420: Retry Generate Name (GA in v1.32 @2024)
- KEP-5116: Streaming Encoding for LIST Responses(Beta in v1.33@2025)
- **KEP-4988** Snapshottable API server cache(Beta in v1.34@2025)
- ...



Kubernetes and etcd Features That Unlocked 15k Node Clusters

Wojciech Tyczyński

Staff Software Engineer, Google



<https://kubernetes.io/blog/2025/02/28/nftables-kube-proxy/>

ETCD 相关更新

- etcd 3.5 improvements further increase transaction concurrency.

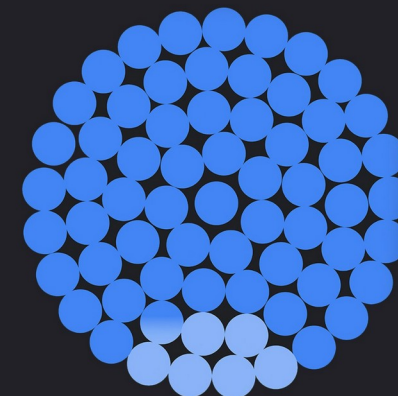
Performance Improvements In Etcd 3.5

Wilson Wang, Jiaxin Shan
Bytedance Infrastructure Computing Team

ByteDance 字节跳动

Support
for up to
65,000
nodes

Google Cloud



Google replaced with Spanner

Notice: The minimum recommended etcd versions to run in production are v3.4.22+ and v3.5.6+.



ETCD 3.6

相比 3.5

- 减少了至少 50% 的平均内存使用：snapshot-count 默认值调整，raft 更频繁的压缩
- v3.6 在读写吞吐量上平均提升约 10%

关键提示：升级前请先升级至 **v3.5.26** 或更高版本，再升级到 v3.6.0，否则升级可能失败。



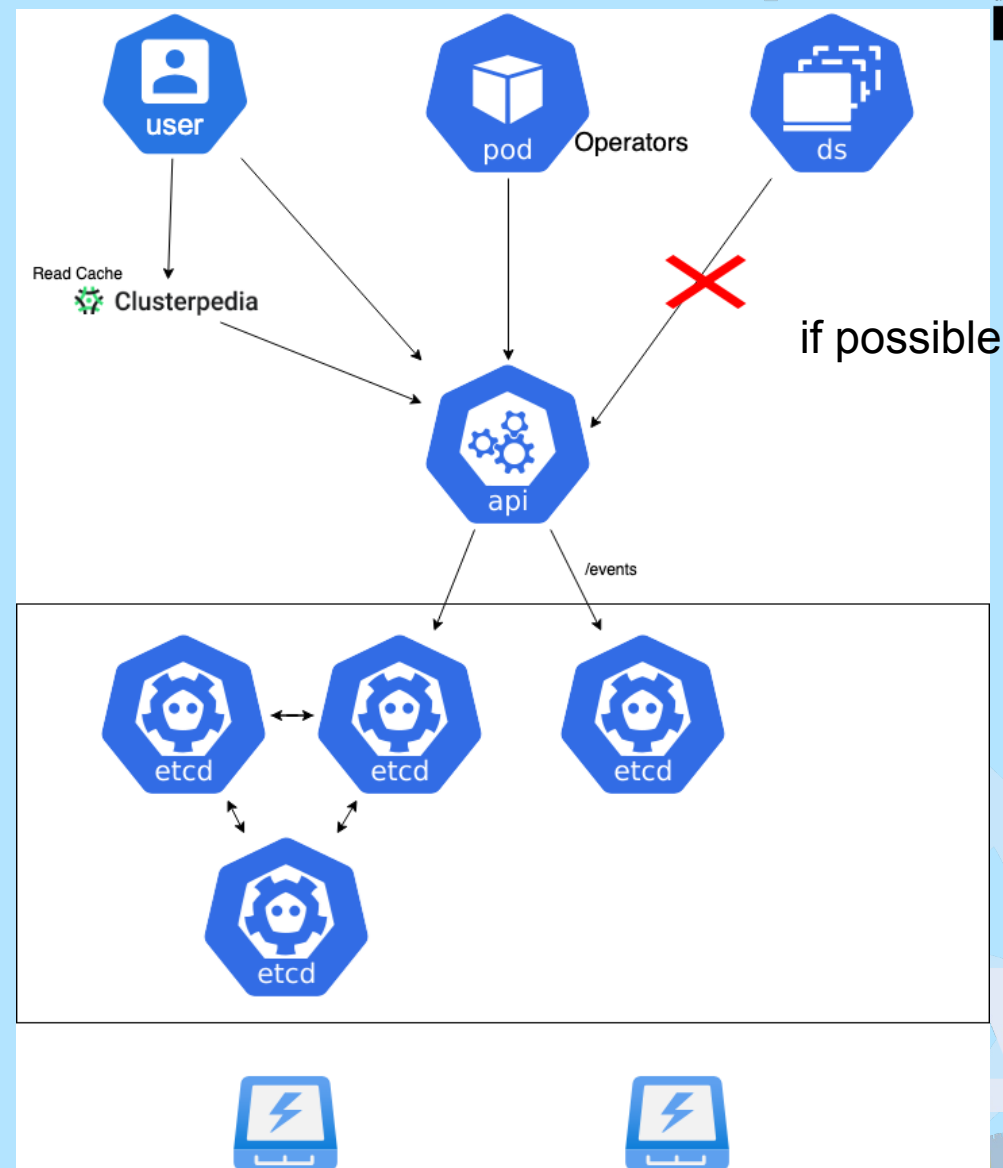
如何降低 ETCD 的压力?

API Server

- 使用 Limit Ranger 和 Resource Quota
- 使用 Clusterpedia 作为读缓存
- 修改某些 Operator : 避免 agent (DS) 直连
- 使用校验策略 (OPA / Kyverno / Validation Admission Policy)
- 对 events 使用 --etcd-servers-overrides
- 优化 Kubelet 配置
- 调整 kubeAPIQPS 和 kubeAPIBurst (v1.27)
- 调整节点更新频率 (Node Lease)

ETCD

- 网络
- 存储
- Heartbeat 间隔



ETCD 替代方案：KubeBrain



kubewharf/kubebrain 开源版本支持 TiKV. 22 年已经支持 2 万节点集群。

- KubeBrain 的维护者去年 2025 年 6 月于香港举办的 KubeCon + CloudNativeCon China 上介绍了该项目
 - An Alternative Metadata System for Large Kubernetes Clusters - Yingcai Xue & Yixiang Chen, ByteDance

当前限制

- Kubernetes 仅支持到 v1.25 及以下
- API Server：如果配合一个专用补丁，性能还可以进一步提升（计划后续开源）
- 该项目已在 ByteDance 内部使用，并且在内部维护得较好，但社区活跃度并不高
- 后端目前仅支持 memkv、TiKV 和 Badger

期待他们未来的进一步更新。



ETCD 替代方案



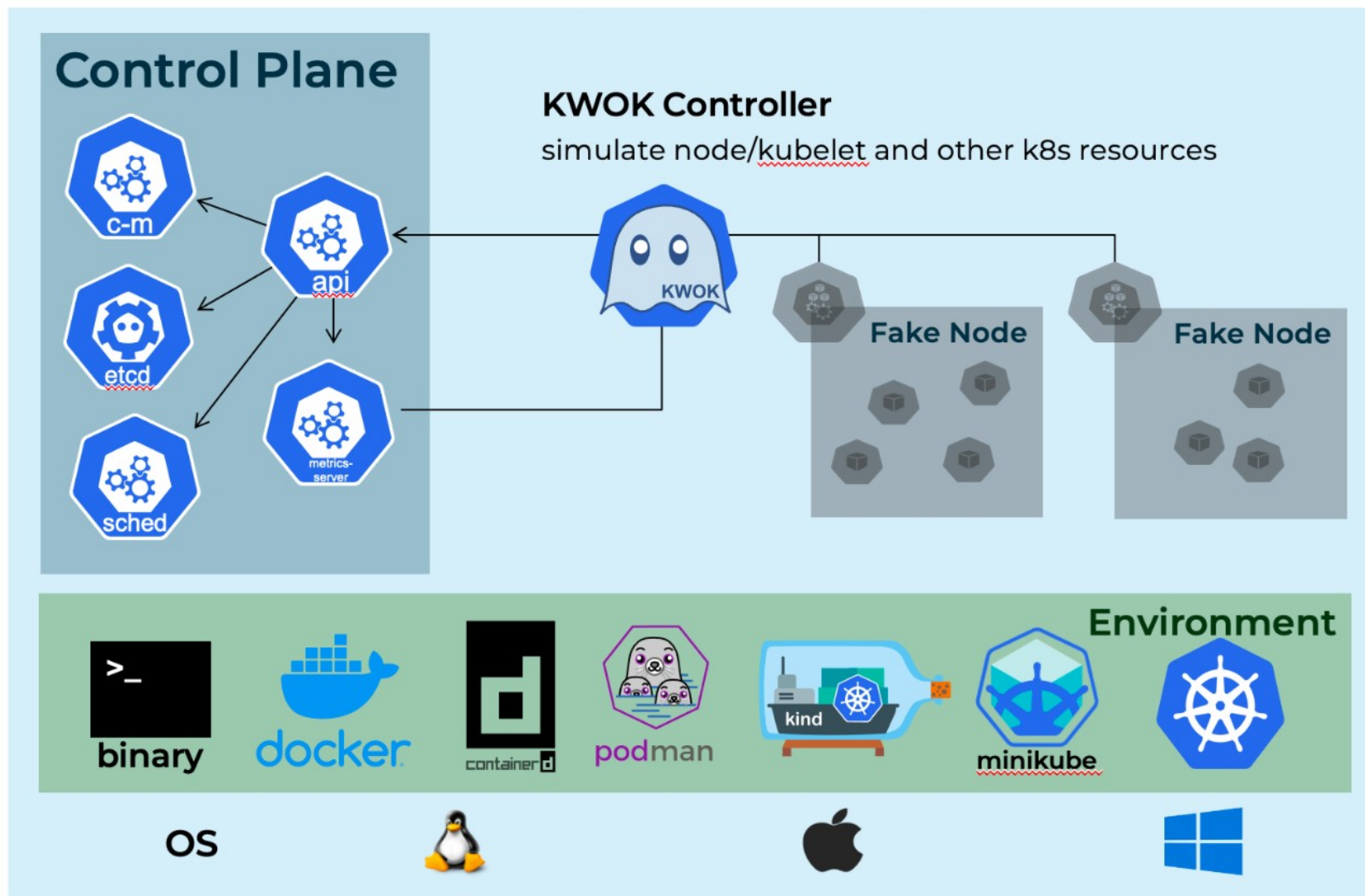
- Google Spanner
- Google Spanner can support clusters of up to 65,000 nodes.



- k3s-io/kine
- A lightweight shim layer that translates the ETCD API to other database backends. Commonly used in edge computing scenarios, such as K3S.
- Supports SQLite、Postgres、MySQL/MariaDB、NATS



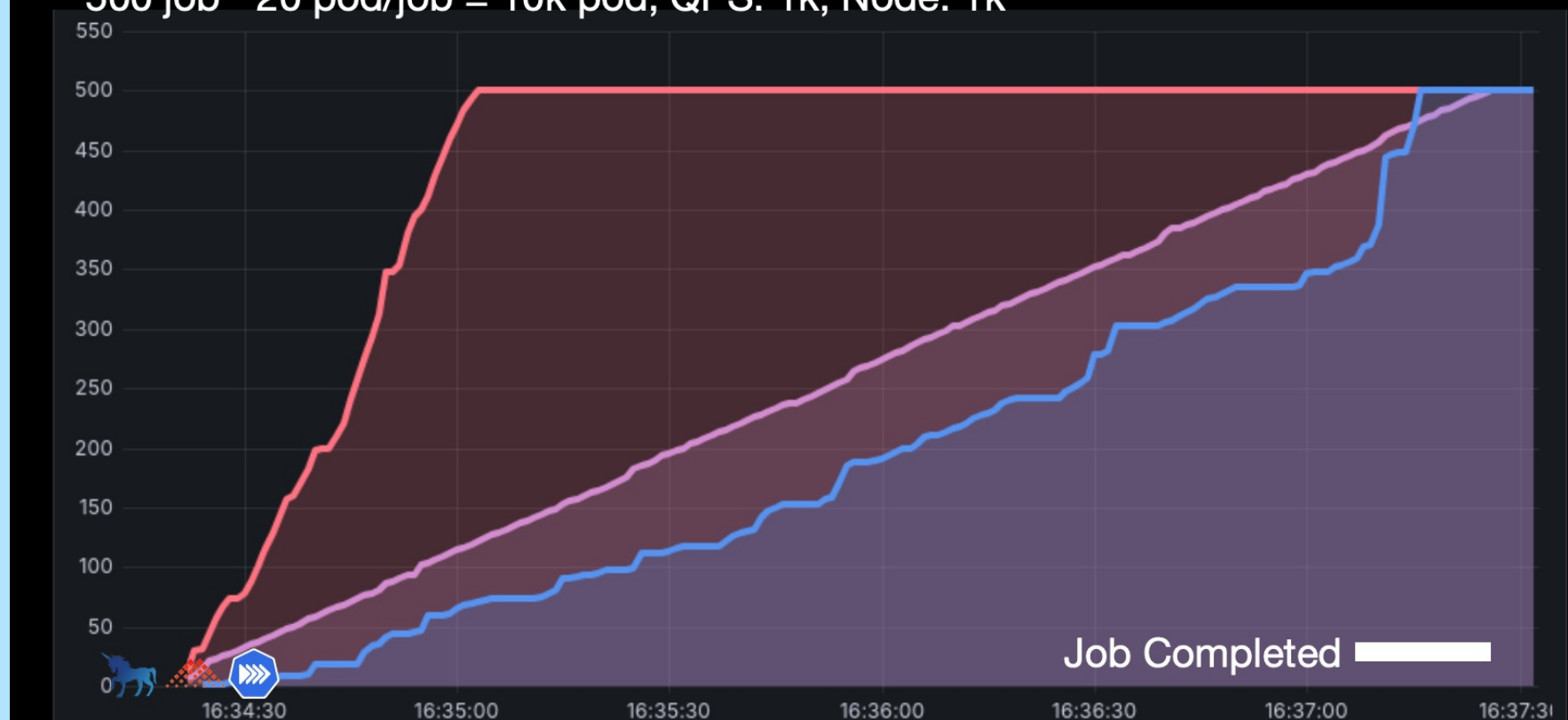
如何测试大规模集群



 **kwokctl**
command line tool

Job Submission with Gang Scheduling

500 job * 20 pod/job = 10k pod, QPS: 1k, Node: 1k



: [A Comparative Analysis of Kueue, Volcano, and YuniKorn - Wei Huang, Apple & Shiming Zhang, DaoCloud](#)

你准备好构建超大规模集群了吗？

Key Advice / 关键建议

- 关注 etcd、scheduler 等核心组件的替代方案
- 跟进 Kubernetes 与 ETCD 近几个版本的性能优化进展
- 判断这些优化是否能解决你的实际场景问题
- 在上线前，对 control plane 和 operators 做系统性的性能压测

还有哪些值得继续探讨的话题？

- 多租户
- 多集群



Part 03

多集群 / 多租户

多集群 / 多租户的常见痛点



为什么不采用多个小集群?

优点

- 隔离性强
- Kubernetes 版本可灵活定制
- 集群内部拥有完整的运维自主权
- 节点配置更加灵活
- 故障影响范围较小
- 可以使用不同版本的 CRD

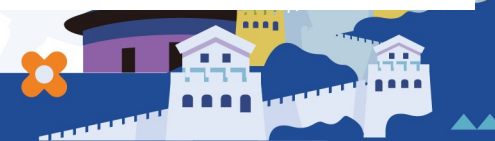
缺点

- 运维成本更高 (可能需要借助 Operator)
- 维护开销更大
- 对子集群的控制力较弱 (或者需要各子集群自行管理)
- 需要多层调度器来解决负载不均和资源碎片化问题
- 成本持续上升

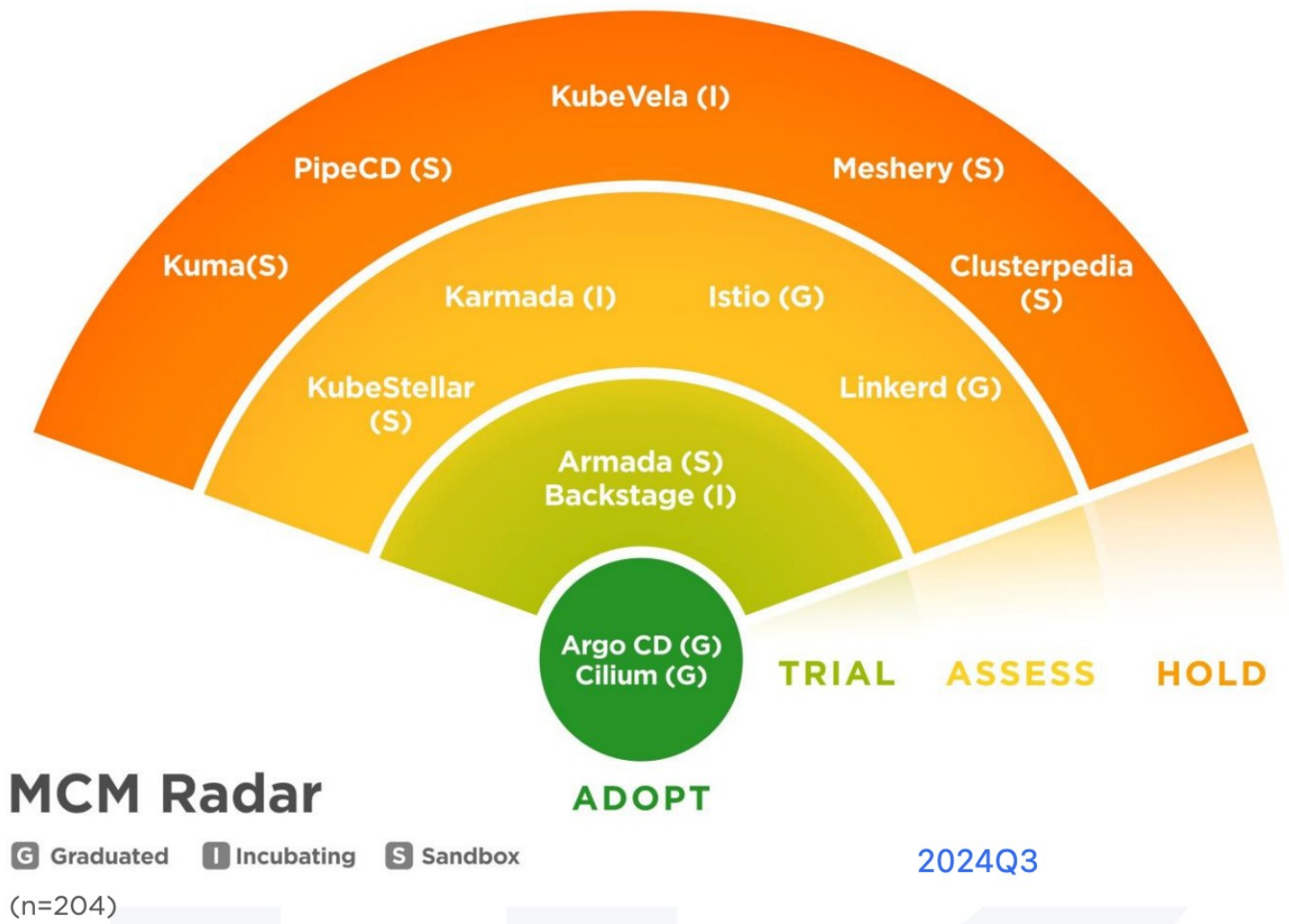


Control plane costs

Platform stack costs



多集群应用管理



<https://www.cncf.io/reports/cncf-technology-landscape-radar/>

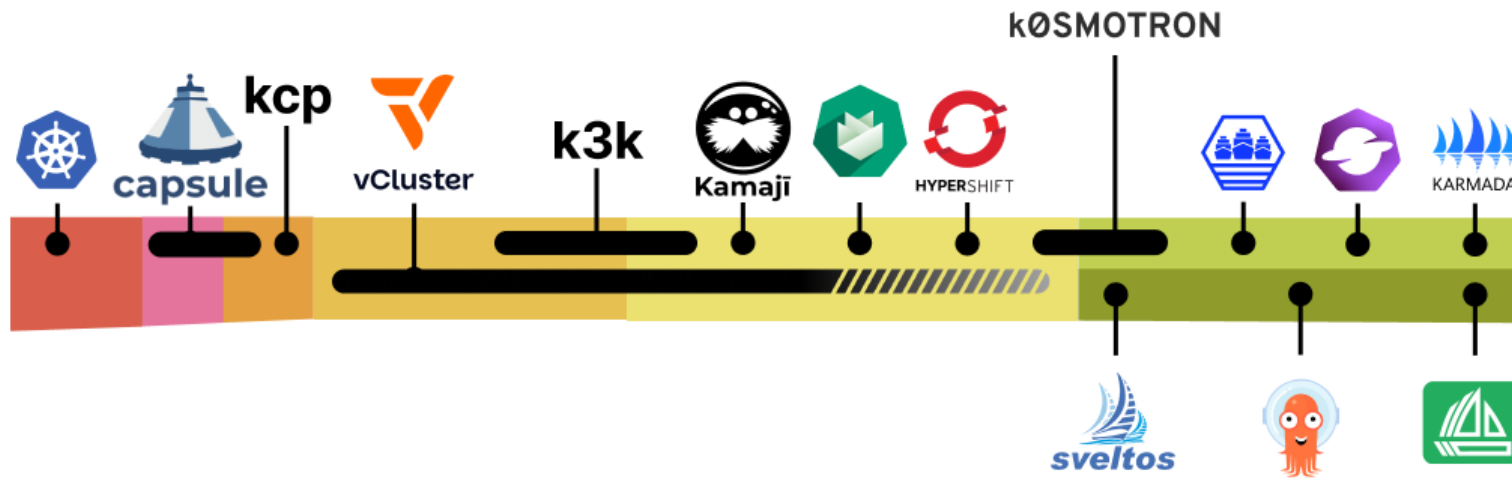


Namespace 方案的几个核心问题

- 租户一多，管理复杂度迅速上升
- 团队级 kubeconfig 分发与权限控制困难
- CRD 等集群级资源的访问边界不清晰
- 资源共享冲突依然存在
- 难以兼容不同 Kubernetes 版本诉求
- 难以满足不同应用版本并存需求

本质上仍是单控制面、单集群状态





- Namespaces
- Namespaces as a Service
- Kubernetes API as a Service
- Control plane as a service (internal)
- Control plane as a service (external)
- Dedicated clusters

learnk8s

ArgoCD, Sveltos, Kubeapps, Drone, Buildpacks etc

vNode, Gatekeeper, Vault, Trivi, Keycloak,
KubeClarity, Cert-manager, Oauth2 etc.

Prometheus, Loki, Grafana, Kiali, Jaeger, Thanos, etc.

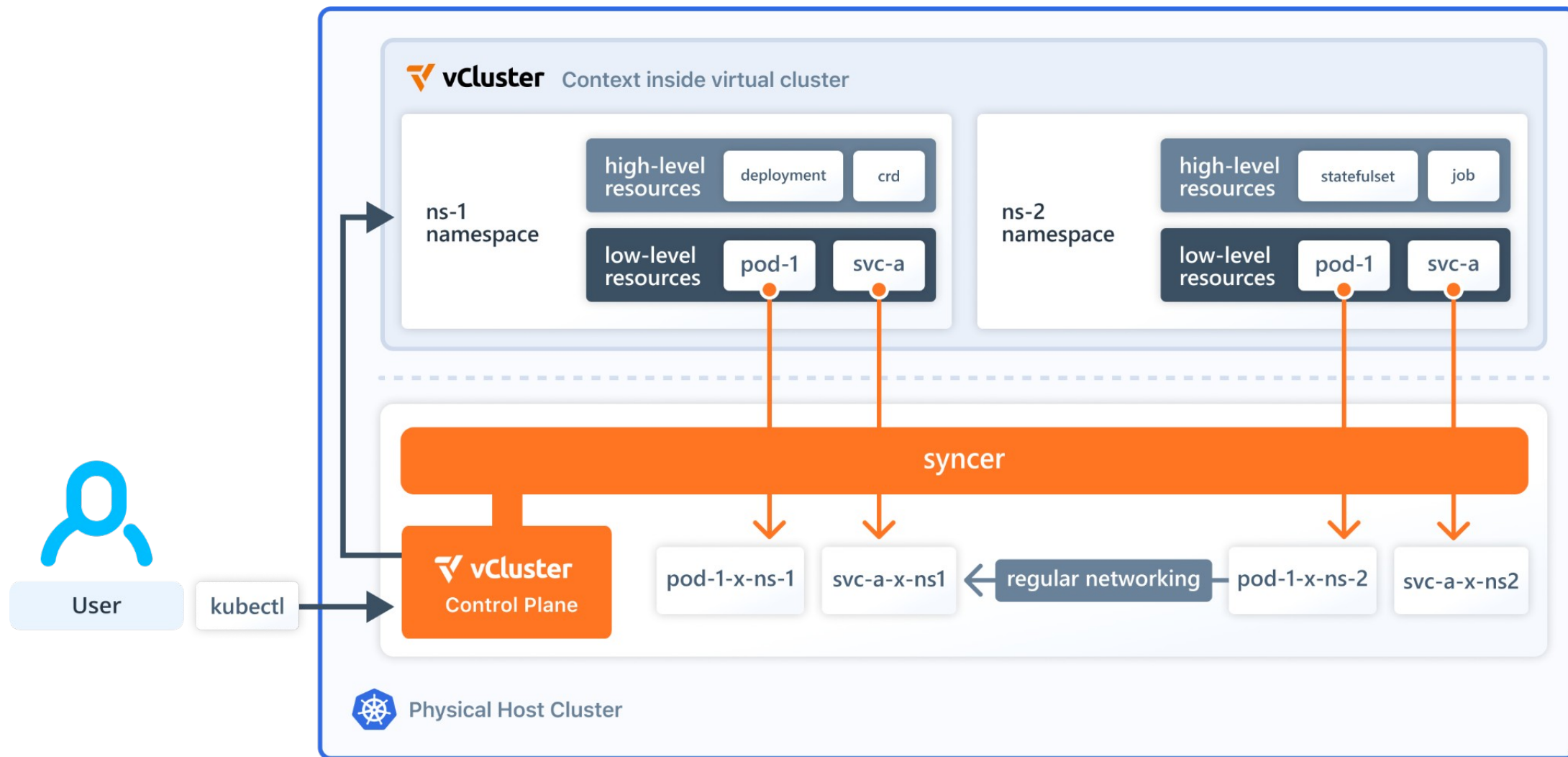
vCluster

Calico, Flannel & VPC-CNI | Istio, Linkerd,
AppMesh & Nginx | Minio, Velero

Kubernetes Cluster, NodeGroups, Nodes



vCluster Architecture



Part 04

Networking: Multi-Cluster or Large Cluster?



Agenda

01

Single-Cluster Networking Problems

Network planning, performance scaling, DNS, fault domains

02

Why Multi-Cluster?

Benefits, challenges, and the transition from scaling to redesign

03

Multi-Cluster Networking Solutions

Submariner, Cilium, Istio, Clusternet, Karmada, Ligo



01

Single-Cluster Networking Problems

Network planning, performance, DNS, and fault domains at scale



Upfront Network Planning

Most large-cluster networking problems are decided before the cluster is even built.

Route Model

- Per-node routes hit VPC limits first
- Route table size & cloud quotas
- Management complexity grows fast

IP & CIDR Planning

- Node, Pod, Service address spaces
- Sufficient Pod IPs per node?
- Multi-AZ / multi-DC subnet design
- Future expansion capacity

The later you plan your network, the higher the remediation cost.



What Happens Without Planning?



Routes explode

VPC route tables hit quota limits before the control plane does



Pod IPs run out

New pods can't schedule even though nodes are available



Subnets exhausted

Some AZs run dry while others have spare capacity



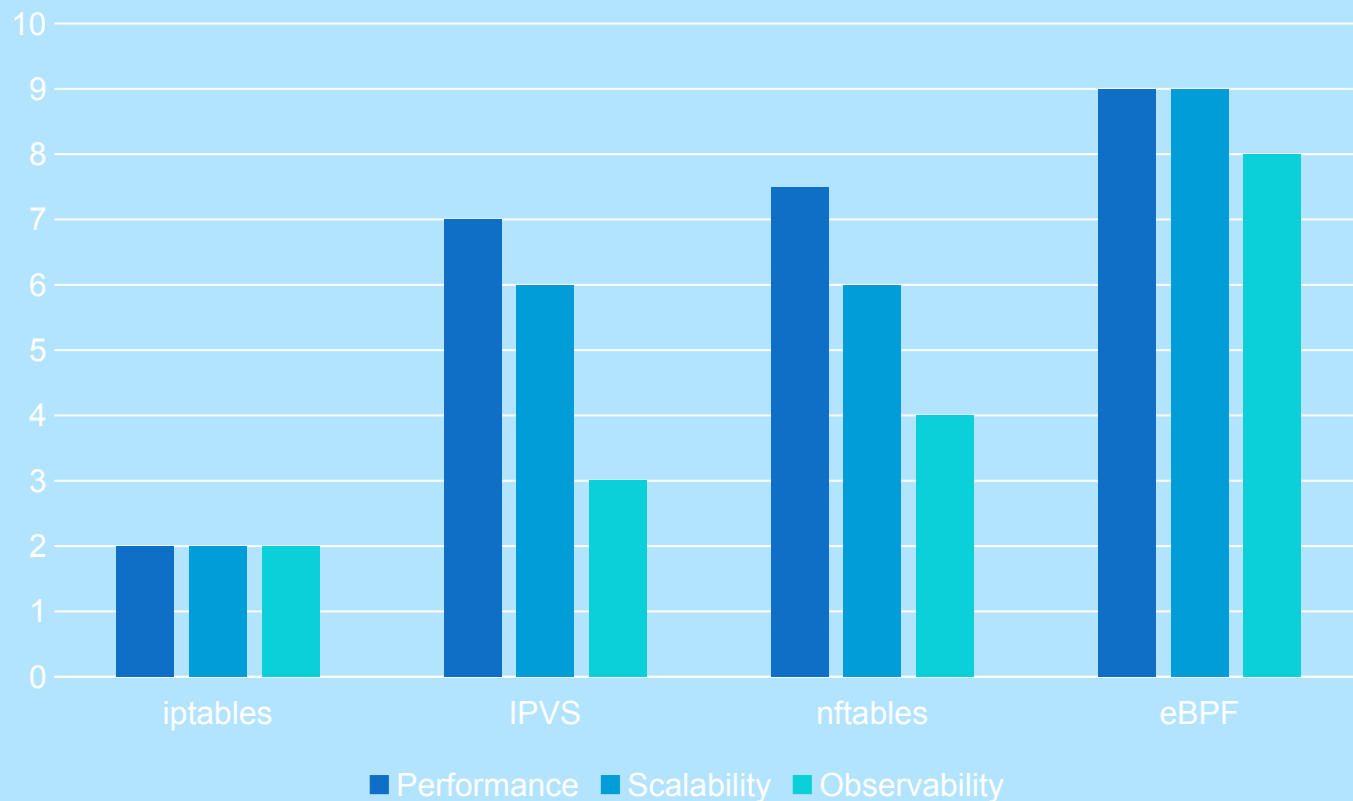
Forced re-architecture

Migrate, split clusters, or rebuild the network from scratch

Biggest fear at scale: not running out of machines, but running out of IPs and routes.



Kube-proxy: Service Data Plane Evolution



iptables

Long rule chains, costly sync

nftables

Next-gen packet filtering, better rule organization

eBPF

Shortest path, best scalability, higher ops bar

IPVS

Historical path; community focus shifted

iptables → nftables / eBPF is not just a speed upgrade — it's a scalability model change.

Cross-Node Traffic: Overlay vs Underlay

LLM

As cluster scale grows, east-west traffic ratio rises sharply.

Overlay

- Easier cross-env migration
- Multi-cloud friendly
- Encapsulation overhead & MTU issues
- Harder troubleshooting

Underlay / Routed

- Better raw performance
- Closer to physical network
- Heavier CIDR planning
- Tighter environment constraints

The real bottleneck is often not packet forwarding speed,
but whether the network model can sustain continued growth.

DNS & Service Discovery + Fault Domain Expansion

DNS & Service Discovery

- CoreDNS QPS grows sharply
- Short-lived connections stress cache
- Headless Services amplify tail latency
- Sidecar discovery adds more load

At scale, service discovery IS the network.

Fault Domain Expansion

- CNI misconfig affects all nodes
- Bad NetworkPolicy rolls globally
- DNS / gateway jitter cascades
- Local network issue → global impact

This is why teams move to multi-cluster.

IP planning must happen upfront. Pod IPs, subnets, expansion paths — all before Day 1.

02

Why Multi-Cluster?

Not just resource splitting — it's network architecture redesign



Multi-Cluster: Benefits & Challenges

Redesigning boundaries:

Fault Domain

Geography

Compliance

Tenancy

Performance

Benefits

- Split network scale pressure
- Shrink blast radius
- Clearer isolation by region/tenant/env
- Independent network planning per env

Challenges

- How to interconnect clusters?
- Cross-cluster performance guarantees?
- Security across trust boundaries?
- Unified service discovery & naming?
- Will ops complexity increase?

Multi-cluster is not resource splitting — it's network architecture redesign.

First: What Level of Connectivity?

Pod-to-Pod

Full flat network across clusters

Service-to-Service

Service-level access without full mesh

Gateway-to-Service

Ingress/Gateway routes to remote services

Key questions before choosing:

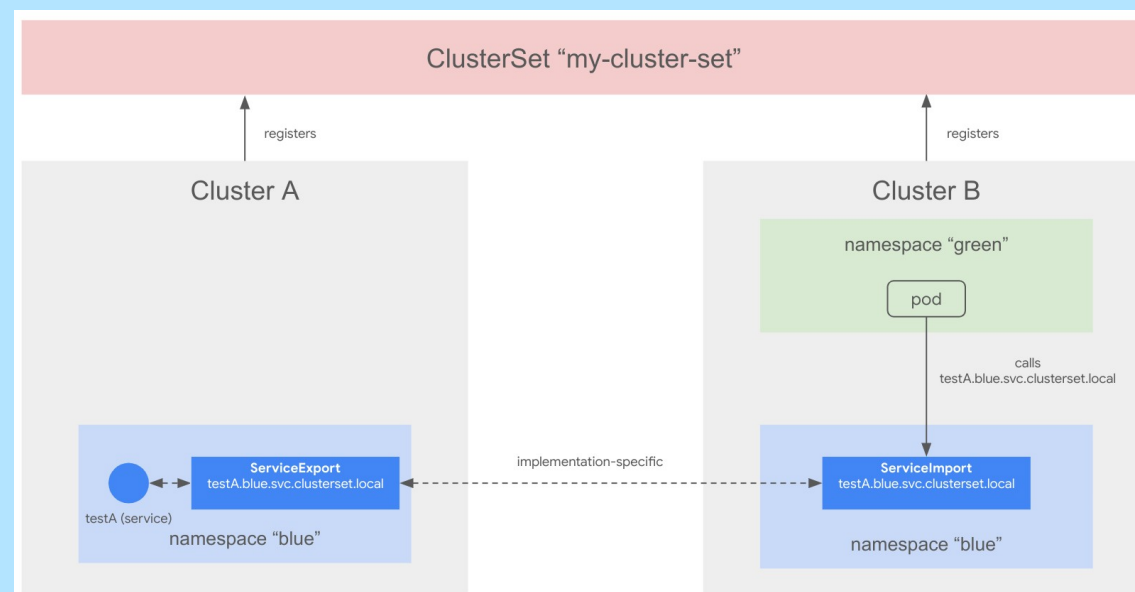
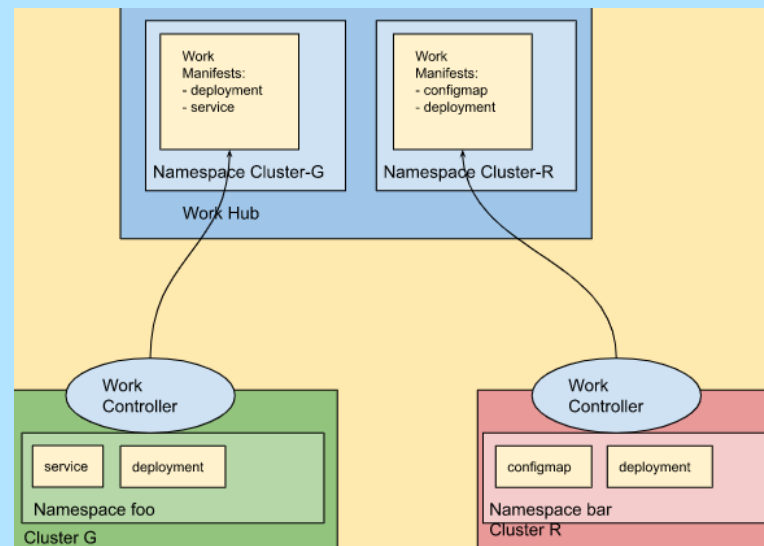
- Do you need full Pod CIDR reachability?
- Is NAT acceptable? Source IP preservation?
- Bidirectional or unidirectional access?



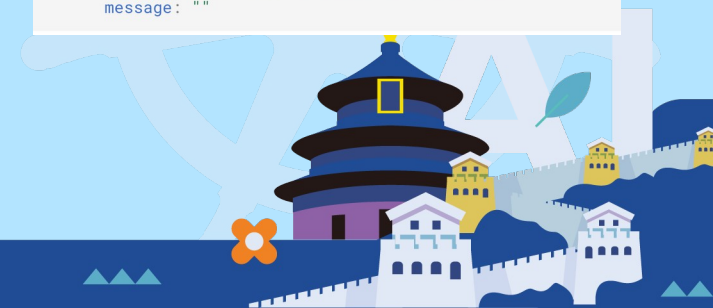
SIG MultiCluster

Three Pillars

- Work API
- Multicluster Services API
- ClusterProfile API



```
apiVersion: multicluster.x-k8s.io/v1alpha1
kind: ClusterProfile
metadata:
  name: some-cluster-name
  namespace: fleet-system
  labels:
    x-k8s.io/cluster-manager: some-cluster-manager
spec:
  displayName: some-cluster
  clusterManager:
    name: some-cluster-manager
status:
  version:
    kubernetes: 1.28.0
  properties:
    - name: clusterset.k8s.io
      value: some-clusterset
    - name: location
      value: apac
  accessProviders:
    - name: secretreader
      cluster:
        server: https://api.some-cluster.example:6443
        certificate-authority-data: <base64-encoded-ca>
        extensions:
          - name: client.authentication.k8s.io/exec
            extension:
              clusterName: some-cluster-name
  conditions:
    - type: ControlPlaneHealthy
      status: True
      lastTransitionTime: "2023-05-08T07:56:55Z"
      message: ""
    - type: Joined
      status: True
      lastTransitionTime: "2023-05-08T07:58:55Z"
      message: ""
```



03

Multi-Cluster Network-Layer Solutions



Submariner

Connecting clusters first



Main Capabilities

- Cross-cluster network connectivity
- Pod / Service communication across clusters
- Lighthouse-based service discovery
- Focus: connect clusters first, then add service discovery
- Architecture: gateway / tunnel plus Lighthouse
- Best for: hybrid cloud, multi-VPC, connectivity-first teams

Limitations / Challenges

- Requires network planning across clusters, ops Complexity
- Performance depends on tunnel and gateway path design
- Policy and observability are not as integrated as Cilium
- Still needs operational work for gateways, NAT, and routing

Key message: Submariner reduces the multi-cluster problem to connectivity first.

Cilium Cluster Mesh

High-performance multi-cluster networking



Main Capabilities

- Multi-cluster Pod / Service connectivity
- Global service discovery
- eBPF-based high-performance dataplane
- Unified policy and observability
- Architecture: sync endpoints and enforce via eBPF dataplane
- Best for: teams already standardized on Cilium

Limitations / Challenges

- Usually requires a Cilium-based environment end to end
- Operational complexity is higher than simple tunnel-based solutions
- Cross-cluster identity, policy, and troubleshooting demand stronger expertise
- Less suitable if you only need basic connectivity quickly

Key message: Cilium Cluster Mesh brings performance, policy, and observability into multi-cluster networking.

Clusternet

Multi-cluster application governance



Clusternet



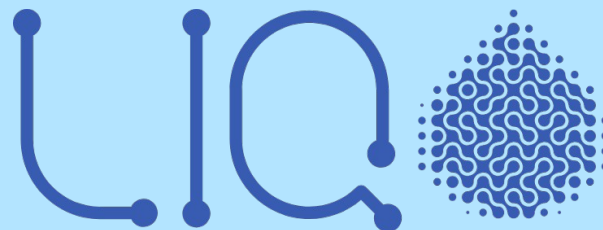
Main Capabilities

- Multi-cluster application distribution
- Centralized governance and lifecycle management
- Policy-driven placement across clusters
- Focus: the control tower for many clusters
- Architecture: central control plane distributing workloads and policies
- Best for: large fleets needing governance and rollout consistency

Limitations / Challenges

- It does not solve network transport by itself
- Usually needs to be paired with a separate network-layer solution
- Introduces an additional control plane and governance model
- Not ideal if your primary pain point is low-level connectivity

Key message: Clusternet is a governance layer for many clusters, not a standalone networking fabric.



Main Capabilities

- Cross-cluster resource sharing
- Virtualized resource consumption
- Integrated network and identity abstraction
- Elastic capacity borrowing across clusters
- Architecture: peer clusters exchange network, identity, and resource abstractions
- Best for: elastic scale-out and resource pooling

Limitations / Challenges

- Concept and operations can feel heavier than simple service export
- Requires trust, identity, and network integration between clusters
- May be more than needed if the goal is only cross-cluster service access
- Adoption depends on whether resource sharing is really the main requirement

Key message: Liqo is about sharing resources across clusters, not only connecting services.



Solution Comparison

Feature	Submariner	Cilium Cluster Mesh	Clusternet	Liqo
Core Positioning	Cross-cluster L3 Connectivity	High-performance eBPF Networking	Multi-cluster App Management	Seamless Cluster Expansion
Technical Core	IPsec / WireGuard Tunnels	eBPF + Direct Routing/Overlay	WebSocket/TCP + MCS API	Virtual Kubelet (Abstraction) + WireGuard Tunnel
Pod Connectivity	Full (Flat L3 network)	Full (Native performance)	Limited (Relies on underlay)	Full(Overcoming Overlapping CIDRs)
CNI Dependency	Agnostic (Works with any CNI)	Exclusive (Requires Cilium)	Agnostic	Agnostic
Service Discovery	Lighthouse (MCS-compliant)	Native ServiceMesh-like	Standard MCS API Support	Dynamic Proxy & Reflection
Security	Tunnel Encryption (IPsec)	Fine-grained eBPF Policies	Certificate-based Auth	Identity Federation
Setup Complexity	Moderate (Requires Gateways)	Low (If already using Cilium)	Low (Lightweight Agents)	Very Low (Zero-config Peering)
Primary Use Case	Interconnecting existing clusters	High-performance Cloud Native	Edge Computing / Fleet Mgmt	Hybrid Cloud / Cloud Bursting

Thanks.

